



Faculté des Sciences et Techniques

RAPPORT DE PROJET

Principes des Systèmes d'Exploitation

Licence 2 – Info 4B – Semestre 4

Lode Runner

Jeu de plates-formes multijoueur en réseau

Réalisé par :

Zenedine EL KARAFI
Mohamed Kazaliou BAH

Année 2025–2026

Note sur l'utilisation du LLM : conformément aux règles du sujet, les passages du rapport rédigés avec l'assistance d'un modèle de langage sont indiqués **en rouge**. Les détails des prompts et des lignes de code concernées figurent en annexe A.

Contents

1	Introduction et analyse fonctionnelle	3
1.1	Présentation du jeu	3
1.2	Règles de fonctionnement	3
1.3	Modes de jeu	3
1.4	Contraintes techniques	3
1.5	Découpage en sous-problèmes	4
2	Architecture logicielle en couches	4
2.1	Principe	4
2.2	Détail de chaque couche	5
2.2.1	Couche 1 — Utilitaires (analogue aux pilotes de périphériques)	5
2.2.2	Couche 2 — Modèle (analogue à la gestion de la mémoire)	5
2.2.3	Couche 3 — Moteur de jeu (analogue au noyau / ordonnancement)	5
2.2.4	Couche 4 — Réseau (analogue à la gestion des E/S inter-processus)	6
2.2.5	Couche 5 — Interface utilisateur (analogue au shell / GUI)	6
3	Structures de données	6
3.1	La grille : <code>Case[] []</code>	6
3.2	Listes d'entités : <code>ArrayList</code>	6
3.3	File BFS : <code>LinkedList</code> comme <code>Queue</code>	6
3.4	Trous en attente : <code>ArrayList<TrouEnAttente></code>	6
3.5	Cache d'images : <code>HashMap<String, BufferedImage></code>	7
3.6	Leaderboard : <code>List<EntreeLeaderboard></code>	7
3.7	Protocole réseau : sérialisation Java	7
3.8	Format des fichiers de niveaux	7
4	Spécification des classes principales	7
4.1	<code>Plateau</code>	7
4.2	<code>Entite</code> (abstraite)	8
4.3	<code>Physique</code>	8
4.4	<code>IA</code>	8
4.5	<code>Moteur</code>	9
4.6	<code>Serveur</code>	9
4.7	Diagramme de classes simplifié	9
4.8	Justification des membres <code>static</code>	10
5	Algorithmes principaux	10
5.1	Boucle de jeu à cadence fixe	10
5.2	BFS de l'IA	11
5.3	Régénération des trous	11
5.4	Détection des collisions	11
5.5	Génération procédurale du niveau 3	12
5.6	Protocole réseau (diagramme de séquence)	12
5.7	Gestion de la concurrence	12

6	Jeu de tests	13
6.1	Mode solo	13
6.2	Mode réseau	14
6.3	Génération procédurale	16
7	Répartition des tâches	16
8	Conclusion	16
A	Utilisation d’outils LLM	16

1 Introduction et analyse fonctionnelle

1.1 Présentation du jeu

Lode Runner est un jeu de plates-formes développé par Douglas Smith et publié par Brøderbund en 1983. Le joueur évolue dans un tableau composé d'échelles, de murs de briques et de passerelles. Son objectif est de ramasser tous les lingots d'or dispersés dans le tableau, puis de rejoindre la sortie via l'échelle principale.

Des gardes patrouillent le tableau et tentent d'attraper le joueur. Pour se défendre, le joueur peut creuser des briques à sa gauche ou à sa droite. Les trous se rebouchent après quelques secondes. Un garde enseveli est réinitialisé à sa **position de départ** une fois le trou rebouché (implémentation choisie : `garde.reset()`, voir section 5.4).

1.2 Règles de fonctionnement

- Le joueur commence avec **5 vies** (`Joueur.java`, constructeur).
- Ramasser un lingot rapporte **10 points** (instruction `ajouterScore(10)` dans `Moteur.update()` et `Serveur.update()`).
- Atteindre la sortie rapporte **100 points** supplémentaires.
- La sortie n'est accessible que lorsque **tous les lingots** ont été collectés (`Plateau.tousLesLingotsRecoltes` vérifié dans `Physique.peutSeDeplacer()`).
- Une collision avec un garde fait perdre **une vie** ; le joueur et le garde sont réinitialisés à leur position de départ (`reset()`).
- Si toutes les vies sont perdues, la partie s'arrête (*Game Over*).
- Le joueur creuse un mur en appuyant sur **Espace** dans la direction courante. La cible est la case MUR à (`joueurX + dx`, `joueurY + 1`).
- Un trou se rebouche après **80 ticks** (= 4 secondes à 20 FPS, constante `TEMPS_REBOUCHAGE` dans `Regenerateur.java`).
- Un garde tombé dans un trou est marqué `bloque = true` et ne peut plus se déplacer.
- Un garde bloqué dans un trou **ne provoque pas de collision** avec le joueur (condition `Case.TROU` dans `verifierCollisions()`).
- Quand un trou se rebouche, `Garde.reset()` replace le garde à sa position initiale et remet `bloque = false` (sinon le garde resterait figé indéfiniment).
- La gravité s'applique à chaque tick via `Physique.doitTomber()` et `Entite.tomber()`.
- Le joueur peut se déplacer latéralement même en chute libre.
- Les passerelles (`Case.PASSERELLE`) soutiennent les entités sans les bloquer horizontalement.

1.3 Modes de jeu

Mode solo Un joueur humain contre des gardes contrôlés par l'IA (`GestionnaireNiveaux`).

Mode coopératif réseau Plusieurs joueurs humains collectent les lingots ensemble. Chaque joueur peut indiquer un **nom d'équipe** ; les scores sont cumulés par équipe (`Serveur.calculerScore`).

Mode combat réseau Les gardes sont contrôlés par des joueurs humains connectés avec le rôle `RoleJoueur.GARDE` (`MessageProtocol.java`).

1.4 Contraintes techniques

Le projet est développé en **Java 17** avec Maven comme outil de build (`pom.xml`). Il a été réalisé et testé sur **GNU/Linux** (Fedora, compatible Debian ≥ 12) et est compilable sur les machines des salles informatiques via :

```
mvn compile
mvn exec:java -Dexec.mainClass="loderunner.Main"           # mode solo
mvn exec:java -Dexec.mainClass="loderunner.network.Serveur" # serveur
    reseau
mvn exec:java -Dexec.mainClass="loderunner.network.Client"  # client
    reseau
```

Conventions Java : le code respecte les conventions d'écriture Java (<https://www.jmdoudoux.fr/java/dej/chap-normesdev.htm>) : noms de classes en PascalCase, méthodes et variables en camelCase, constantes en SCREAMING_SNAKE_CASE, commentaires Javadoc sur les méthodes publiques.

Ordre de développement : conformément aux recommandations du sujet, le jeu a d'abord été développé et testé en **mode solo** (moteur, physique, IA, interface graphique) avant d'implanter le mode réseau. Les tests T1 à T13 de la section 6 valident le fonctionnement en mode machine unique.

1.5 Découpage en sous-problèmes

1. **Représentation du monde :** grille `Case[][]`, entités (`model.*`).
2. **Physique :** gravité, déplacement, creusage (`Physique`).
3. **Intelligence artificielle :** BFS vers le joueur (`IA`).
4. **Régénération :** rebouchage temporisé des trous (`Regenerateur`).
5. **Moteur de jeu :** boucle à cadence fixe, coordination (`Moteur`).
6. **Chargement des niveaux :** lecture fichier texte + génération procédurale (`LevelLoader`, `GestionnaireNiveaux`).
7. **Interface graphique :** rendu Swing, sprites, HUD (`GamePanel`, `MainWindow`, `LeaderboardWindow`).
8. **Saisie clavier :** `KeyListener` vers `Direction` (`InputHandler`).
9. **Réseau :** synchronisation TCP client-serveur (`network.*`).
10. **Persistance :** scores sur disque (`leaderboard.txt`).

2 Architecture logicielle en couches

2.1 Principe

Conformément au chapitre 1 du cours Info4B, l'architecture d'un système d'exploitation est organisée en **couches fonctionnelles superposées** : chaque couche ne communique qu'avec la couche inférieure et en masque les détails (principe d'abstraction). Le cours identifie six niveaux : *interface utilisateur*, *gestion des fichiers*, *gestion des E/S*, *gestion de la mémoire*, *gestion du processeur* et *matériel*.

Notre application transpose ce modèle en cinq couches. Les couches *gestion des fichiers* et *gestion des E/S* du cours sont regroupées en une seule couche utilitaires car, dans le contexte d'un jeu, tous les accès aux périphériques (disque, écran) passent exclusivement par des fichiers et des bibliothèques Java — il n'y a pas de pilote de périphérique à écrire directement.

Le tableau ci-dessous détaille la correspondance entre les packages du projet et les couches du cours.

Couche 5 — Interface utilisateur (analogue : shell / GUI d'un SE)	
<code>ui.MainWindow ui.GamePanel ui.InputHandler ui.LeaderboardWindow</code>	
Couche 4 — Réseau / E/S inter-machines (analogue : protocoles réseau d'un SE)	
<code>network.Serveur network.Client network.GestionnaireClient network.MessageProtocol</code>	
Couche 3 — Moteur de jeu (analogue : noyau / gestion du processeur d'un SE)	
<code>game.Moteur game.Physique game.IA game.Regenerateur game.GestionnaireNiveaux</code>	
Couche 2 — Modèle (analogue : gestion de la mémoire d'un SE)	
<code>model.Plateau model.Entite model.Joueur model.Garde model.Case</code>	
Couche 1 — Utilitaires / E/S fichiers (analogue : pilotes de périphériques d'un SE)	
<code>utils.LevelLoader utils.ImageLoader utils.Direction</code>	

Figure 1: Architecture en couches de Lode Runner, calquée sur le modèle SE (cours Info4B, chapitre 1)

2.2 Détail de chaque couche

2.2.1 Couche 1 — Utilitaires (analogue aux pilotes de périphériques)

Cette couche fournit les services d'entrée/sortie de bas niveau, indépendants de la logique de jeu.

- `LevelLoader` : lit un fichier texte et construit le `Plateau` (méthode statique `loadMap(String)`).
- `ImageLoader` : charge et met en cache les sprites PNG (`HashMap<String, BufferedImage>`). Les membres `static` sont justifiés section 4.8.
- `Direction` : énumération des 5 directions (`HAUT`, `BAS`, `GAUCHE`, `DROITE`, `AUCUNE`).

2.2.2 Couche 2 — Modèle (analogue à la gestion de la mémoire)

Cette couche définit les structures de données du monde du jeu, sans aucune logique de traitement.

- `Case` : enum (`MUR`, `ECHELLE`, `PASSERELLE`, `VIDE`, `LINGOT`, `SORTIE`, `TROU`).
- `Plateau` : grille `Case[largeur][hauteur]` + listes `ArrayList` de joueurs et gardes.
- `Entite` (abstraite) : position, position initiale, direction, nom d'équipe, `reset()`.
- `Joueur` : étend `Entite` ; ajoute score (`int`) et vies (`int`, init 5).
- `Garde` : étend `Entite` ; ajoute boolean `bloque`.

2.2.3 Couche 3 — Moteur de jeu (analogue au noyau / ordonnancement)

Cette couche orchestre tous les traitements et alloue le temps de calcul aux entités à cadence fixe.

- `Physique` : `peutSeDeplacer()`, `doitTomber()`, `peutCreuser()`. Ne modifie jamais l'état du modèle.
- `IA` : calcule le chemin optimal vers le joueur le plus proche par BFS. Elle retourne uniquement le premier pas à effectuer.

- **Regenerateur** : décrémentation des compteurs de trous à chaque tick.
- **Moteur** : boucle à 20 FPS en mode solo (`Runnable`, thread daemon).
- **GestionnaireNiveaux** : chargement, enchaînement et génération procédurale des niveaux.

2.2.4 Couche 4 — Réseau (analogue à la gestion des E/S inter-processus)

Cette couche gère toutes les communications entre machines via des sockets TCP.

- **MessageProtocol** : toutes les classes `Serializable` échangées sur le réseau.
- **Serveur** : héberge la physique, diffuse l'état (`writeUnshared`) 20 fois/s.
- **GestionnaireClient** : un thread par connexion TCP entrante.
- **Client** : reçoit l'`EtatJeu`, envoie `MessageInput` à 20 Hz.

2.2.5 Couche 5 — Interface utilisateur (analogue au shell / GUI)

Cette couche est la seule avec laquelle l'utilisateur interagit directement.

- **GamePanel** : `paintComponent()` redessine plateau + entités + HUD à chaque tick. Sprites animés par alternance de frames (`tick / 8 % 2`).
- **InputHandler** : `KeyListener` maintient la direction courante.
- **MainWindow** : menu principal (solo, créer, rejoindre).
- **LeaderboardWindow** : classement individuel et par équipe en fin de partie réseau.

3 Structures de données

3.1 La grille : `Case[][]`

Tableau 2D indexé `[x][y]` (`x` = colonne, `y` = ligne croissant vers le bas). Accès en $O(1)$, indispensable lors des vérifications à chaque tick.

```
1 private Case[][] cases;    // cases[x][y]
2 private int largeur, hauteur;
```

Listing 1: Plateau.java – déclaration de la grille

3.2 Listes d'entités : `ArrayList`

`ArrayList<Joueur>` et `ArrayList<Garde>` permettent un nombre variable d'entités (connexions dynamiques en mode réseau). La méthode `getEntites()` fusionne les deux listes pour les itérations génériques du moteur.

3.3 File BFS : `LinkedList` comme Queue

```
1 Queue<int[]> file      = new LinkedList<>();
2 boolean[][] visite    = new boolean[largeur][hauteur];
3 Direction[][] premierPas = new Direction[largeur][hauteur];
```

Listing 2: IA.java – structures du BFS

3.4 Trous en attente : `ArrayList<TrouEnAttente>`

Classe interne privée de `Regenerateur` : (`x`, `y`, `tempsRestant`).

3.5 Cache d'images : `HashMap<String, BufferedImage>`

Accès en $O(1)$. Évite les lectures disque répétées à 20 FPS.

3.6 Leaderboard : `List<EntreeLeaderboard>`

`ArrayList` triée par score décroissant (max 10 entrées), persistée dans `leaderboard.txt` (format `<nom> <score>` par ligne). Un second classement `List<EntreeEquipe>` cumule les scores par nom d'équipe.

3.7 Protocole réseau : sérialisation Java

Toutes les classes de `MessageProtocol` implémentent `Serializable`. `EtatJeu` transmet les cases sous forme `int[][]` (`Case.ordinal()`) pour simplifier la désérialisation côté client.

3.8 Format des fichiers de niveaux

Première ligne : `LARGEUR HAUTEUR`. Lignes suivantes : grille, un caractère par case.

Caractère	Signification (et constante <code>Case</code>)
#	Mur de briques creusable (<code>Case.MUR</code>)
H	Échelle (<code>Case.ECHELLE</code>)
-	Passerelle (<code>Case.PASSERELLE</code>)
(espace)	Vide (<code>Case.VIDE</code>)
£	Lingot d'or (<code>Case.LINGOT</code>)
S	Sortie (<code>Case.SORTIE</code>)
P	Position initiale du joueur (crée un <code>Joueur</code> dans <code>LevelLoader</code>)
G	Position initiale d'un garde (crée un <code>Garde</code> dans <code>LevelLoader</code>)
O	Trou préexistant (<code>Case.TROU</code>)

Ce format permet d'ajouter des niveaux sans recompiler. Les niveaux 1 et 2 sont statiques ; le niveau 3 est généré par `GestionnaireNiveaux.genererNiveau()` et écrasé à chaque passage.

4 Spécification des classes principales

4.1 Plateau

Méthode	Description
<code>getCase(x, y) : Case</code>	Accès $O(1)$ à la case <code>[x][y]</code>
<code>setCase(x, y, type)</code>	Modifie une case (creusage, rebouchage, collecte lingot)
<code>estPositionValide(x,y)</code>	<code>estdansLePlateau()</code> ET <code>estMarchable()</code> (non-MUR)
<code>tousLesLingotsRecoltes()</code>	Parcourt la grille ; retourne <code>true</code> si aucun <code>LINGOT</code>
<code>getEntiteAt(x,y)</code>	Cherche dans joueurs puis gardes ; retourne <code>null</code> si vide
<code>ajouterJoueur/Garde</code>	Ajoute à la liste correspondante (appelé par <code>LevelLoader</code>)
<code>getEntites()</code>	Fusionne joueurs + gardes en une seule <code>ArrayList<Entite></code>

Remarque code : `estPositionValide()` appelle `estMarchable()` qui exclut les MUR. Dans `Physique.peutSeDeplacer()`, la condition `caseCible == Case.MUR` est donc redondante avec l'appel à `estPositionValide()` – sans impact fonctionnel.

4.2 Entite (abstraite)

Méthode	Description
<code>deplacer(Direction)</code>	Traduit la direction en <code>deplacer(dx, dy)</code>
<code>deplacer(int dx, int dy)</code>	Déplace si <code>plateau.estPositionValide()</code>
<code>tomber()</code>	Raccourci <code>deplacer(0, 1)</code>
<code>reset()</code>	Repositionne à <code>(x_Initiale, y_Initiale)</code>
<code>setDirection/getDirection</code>	Direction courante (utilisée pour l'animation dans <code>GamePanel</code>)
<code>setNomEquipe(String)</code>	Équipe (mode coopératif) ; null remplacé par ""

Note sur Joueur : la classe possède des méthodes `prendrePaquet()`, `prendreLingot()` (50 pts) et `entrerSortie()` qui ne sont **pas appelées** dans la version finale. Le score est géré directement par `ajouterScore(10)` (lingot) et `ajouterScore(100)` (sortie) dans `Moteur` et `Serveur`.

4.3 Physique

Méthode	Description
<code>peutSeDeplacer(e, dir)</code>	Valide le déplacement (murs, échelles, trous, sortie, superposition)
<code>doitTomber(e)</code>	Retourne <code>true</code> si pas de support (ni échelle ni passerelle sous les pieds)
<code>peutCreuser(x, y)</code>	Cible dans le plateau, non sur les bords, et de type MUR

Règles clés de `peutSeDeplacer()` :

- HAUT : autorisé seulement si case actuelle **ou** case cible est une ECHELLE.
- SORTIE : seulement si `tousLesLingotsRecoltes()`.
- TROU occupé : refusé si une entité est déjà dans le trou cible.
- Garde : ne peut pas se superposer à un autre garde.

4.4 IA

Méthode	Description
<code>calculerMouvement(garde, cible)</code>	Retourne <code>Direction.AUCUNE</code> si garde bloqué, sinon BFS
<code>bfs(sx,sy,tx,ty)</code>	Parcours largeur d'abord ; retourne le premier pas vers la cible
<code>peutSeDeplacer(x,y,dir)</code>	Règles IA : MUR interdit, monter nécessite échelle, latéral nécessite sol
<code>estSupporte(x,y)</code>	Sol : échelle, passerelle, ou case inférieure = MUR/ECHELLE/TROU

Note : `estSupporte()` considère un TROU comme un sol. Sans cela, les gardes se figeraient au-dessus des trous (bug corrigé, voir annexe A).

4.5 Moteur

Méthode	Description
<code>run()</code>	Boucle à 20 FPS ; calcule le temps d'attente résiduel
<code>update()</code>	Régénération → gravité → joueur → gardes → collisions
<code>stop()</code>	Passé <code>encours = false</code> (victoire ou mort)
<code>verifierCollisions()</code>	En solo : vérifie uniquement <code>getJoueurs().get(0)</code> contre tous les gardes libres (non en TROU)

Ordonnancement : le joueur se déplace toutes les 3 frames (`DELAI_MOUVEMENT`), les gardes toutes les 6 (`DELAI_MOUVEMENT_GARDES`). Ce mécanisme simule un ordonnancement pondéré analogue à la gestion du processeur dans un SE.

4.6 Serveur

Méthode	Description
<code>run()</code>	Accepte les connexions TCP (<code>ServerSocket</code> , thread daemon)
<code>lancerBoucleJeu()</code>	Boucle principale : <code>update()</code> + <code>broadcastEtat()</code> à 20 FPS
<code>update()</code>	Même logique que <code>Moteur.update()</code> , appliquée aux inputs des clients
<code>assignerEntite(gc,role)</code>	<code>synchronized</code> ; attribue joueur ou garde au client
<code>broadcastEtat()</code>	<code>writeUnshared()</code> vers chaque <code>GestionnaireClient</code>
<code>enregistrerScores()</code>	Ajoute au leaderboard, trie, limite à 10, sauvegarde
<code>calculerScoresEquipes()</code>	Cumule par <code>nomEquipe</code> ; ignore les joueurs sans équipe

4.7 Diagramme de classes simplifié

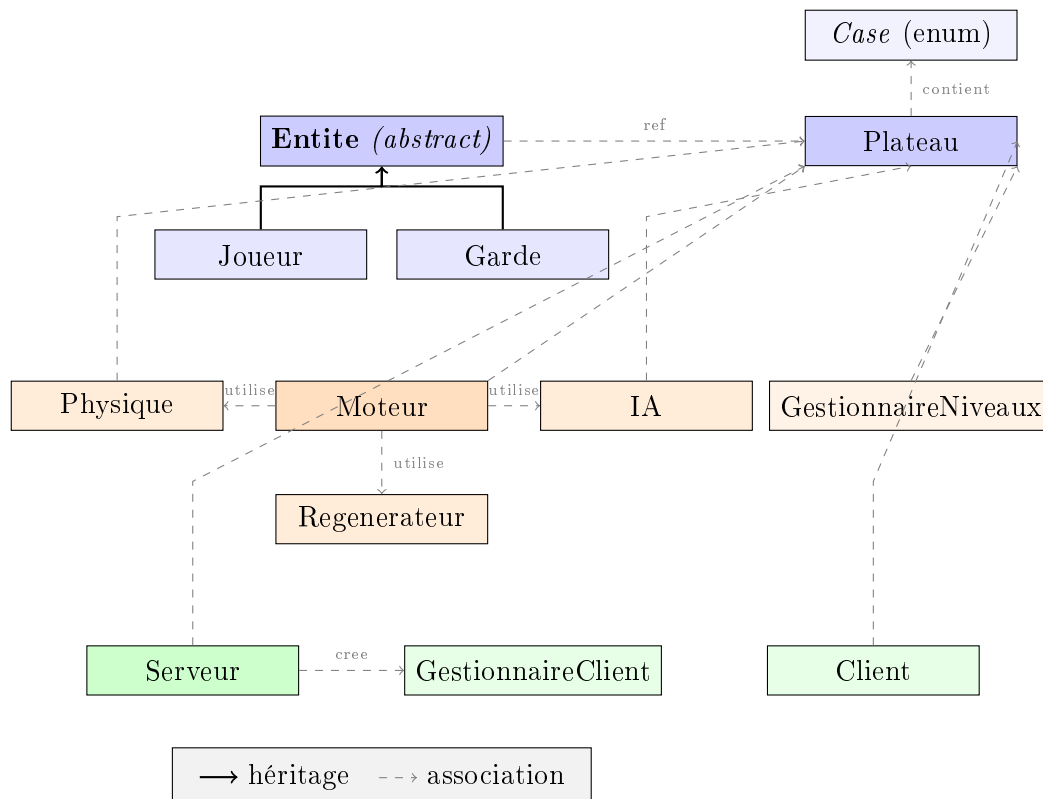


Figure 2: Diagramme de classes simplifié (relations principales)

4.8 Justification des membres static

Le sujet impose de justifier l'emploi de `static`. Voici tous les membres concernés :

Classe	Membre static	Justification
ImageLoader	cacheImages, getImage()	Cache global unique pour toute la JVM (pattern Flyweight). Évite de dupliquer les images entre instances de <code>GamePanel</code> .
MessageProtocol	PORT_DEFAULT	Constante de configuration partagée sans état d'instance.
Moteur	DELAI_MOUVEMENT, DELAI_MOUVEMENT_GARDES	Constantes de cadence identiques pour tous les moteurs.
MainWindow	afficher()	Point d'entrée graphique – aucun état d'instance requis avant création.
LeaderboardWindow	afficher()	Méthode utilitaire appelée depuis <code>Client</code> sans référence sur l'objet.
LevelLoader	loadMap(String)	Service pur sans état, utilisable directement par son nom de classe.

5 Algorithmes principaux

5.1 Boucle de jeu à cadence fixe

```

1 final long DUREE_FRAME = 50; // 20 FPS
2 this.encours = true;
3 while (encours) {
4     long debut = System.currentTimeMillis();
5     update();
6     panel.repaint();
7     long tempsAttente = DUREE_FRAME - (System.currentTimeMillis() - debut);
8     if (tempsAttente > 0) Thread.sleep(tempsAttente);
9 }

```

Listing 3: Moteur.java – boucle principale

Le moteur passe le temps restant de la frame en `sleep()` pour limiter la consommation CPU. En mode réseau, le `Serveur` applique la même boucle puis appelle `broadcastEtat()`.

5.2 BFS de l'IA

Parcours en largeur d'abord sur la grille ; complexité $O(L \times H)$ où $L = 20$ et $H = 15$, soit au plus 300 cases explorées. Seul le *premier pas* depuis la case de départ est mémorisé.

```

1 while (!file.isEmpty()) {
2     int[] pos = file.poll();
3     int x = pos[0], y = pos[1];
4     for (int i = 0; i < directions.length; i++) {
5         if (!peutSeDeplacer(x, y, directions[i])) continue;
6         int nx = x + dx[i], ny = y + dy[i];
7         if (visite[nx][ny]) continue;
8         visite[nx][ny] = true;
9         premierPas[nx][ny] = (x == startX && y == startY)
10             ? directions[i] : premierPas[x][y];
11         if (nx == targetX && ny == targetY)
12             return premierPas[nx][ny];
13         file.add(new int[]{nx, ny});
14     }
15 }

```

Listing 4: IA.java – coeur du BFS

Les cases TROU ne sont pas bloquées dans le BFS (contrairement aux MUR). La chute dans un trou est gérée séparément par `Physique.doitTomber()`.

5.3 Régénération des trous

```

1 Iterator<TrouEnAttente> it = trous.iterator();
2 while (it.hasNext()) {
3     TrouEnAttente t = it.next();
4     t.tempsRestant--;
5     if (t.tempsRestant <= 0) {
6         reboucher(t.x, t.y);
7         it.remove(); // suppression safe (evite
8                     // ConcurrentModificationException)
9     }
10 }

```

Listing 5: Regenerateur.java – rebouchage

5.4 Détection des collisions

```

1 for (Garde g : plateau.getGardes()) {
2     // un garde enseveli ne blesse pas le joueur
3     if (plateau.getCase(g.getX(), g.getY()) == Case.TROU) continue;
4     if (j.getX() == g.getX() && j.getY() == g.getY()) {
5         j.perdreVie();
6         if (j.estMort()) { stop(); return; }
7         resetPositions(); // reset() sur toutes les entites
8     }
9 }

```

Listing 6: Moteur.java – verifierCollisions()

Choix d'implémentation : le `reset()` repose sur la position initiale lue dans le fichier de niveau (`x_Initiale`, `y_Initiale`), et non systématiquement sur le « haut du tableau » comme dans le jeu original.

5.5 Génération procédurale du niveau 3

1. Grille 20×15 vide avec bordures en MUR.
2. 2 à 4 étages horizontaux (murs complets ou percés aléatoirement).
3. Échelle principale pleine hauteur (garantit l'accessibilité de la sortie).
4. Échelles secondaires entre étages adjacents.
5. Passerelles dans les sections libres.
6. Lingots uniquement sur cases soutenues (sol en dessous).
7. Placement joueur, gardes (1 à 3), sortie.
8. Écriture dans `level3.txt` via `BufferedWriter`.

5.6 Protocole réseau (diagramme de séquence)

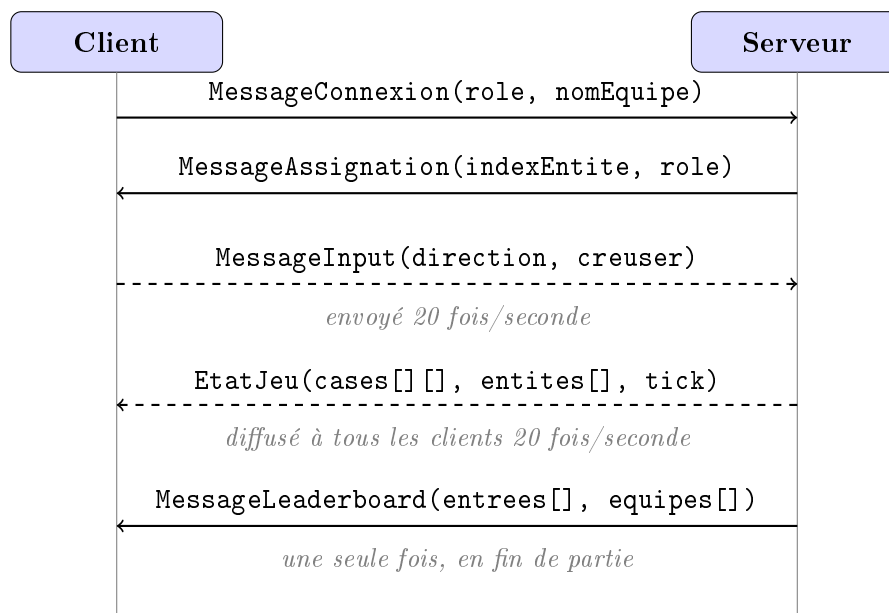


Figure 3: Diagramme de séquence de la communication client-serveur

Point critique : `ObjectOutputStream` doit être créé et `flush()`é avant l'ouverture de l'`ObjectInputStream` des deux côtés, sans quoi un interblocage se produit lors de l'échange des en-têtes de sérialisation.

5.7 Gestion de la concurrence

Plusieurs threads coexistent dans l'application. Le tableau suivant les recense et précise leur rôle, conformément aux concepts d'ordonnancement et de synchronisation étudiés en cours.

Thread	Classe(s)	Rôle
Moteur	Moteur	Boucle de jeu solo à 20 FPS (daemon)
Moniteur	lambda dans <code>GestionnaireNiveaux</code>	Attend la fin du niveau, déclenche le suivant
Accepteur	<code>Serveur.run()</code>	Accepte les connexions <code>ServerSocket</code> (daemon)
Client $\times N$	<code>GestionnaireClient</code>	Un thread par client TCP
Envoyeur	<code>Client.run()</code>	Envoie les inputs à 20 Hz (daemon)

Synchronisation de la liste des clients : la liste `clients` est accédée depuis plusieurs threads simultanément (thread accepteur et boucle de jeu). Pour éviter les conditions de course, tout accès passe par un bloc `synchronized` suivi d'une copie défensive :

```

1 List<GestionnaireClient> clientsCopie;
2 synchronized (clients) {
3     clientsCopie = new ArrayList<>(clients);
4 }
5 // iteration sur clientsCopie : pas de risque meme si un client se
   // deconnecte

```

Listing 7: Serveur.java – copie défensive sous verrou

Architecture *authoritative server* : le serveur est la seule source de vérité pour la physique. Les clients se contentent d'afficher l'`EtatJeu` reçu et d'envoyer leurs inputs. Cette architecture garantit la cohérence de l'état du jeu même en présence de latence réseau variable.

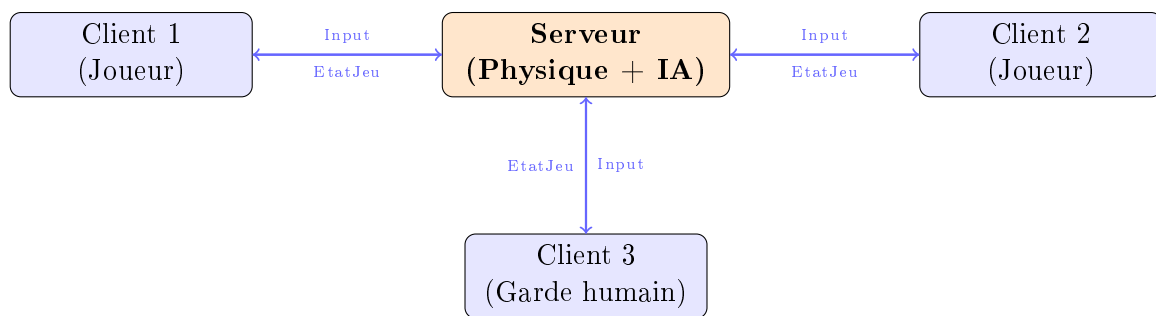


Figure 4: Architecture client-serveur : le serveur est la seule source de vérité

6 Jeu de tests

6.1 Mode solo

Test	Action / condition	Résultat attendu
T1	Passer sur un lingot	Score +10, case → VIDE
T2	Aller vers un MUR	Joueur immobile
T3	Aucun support sous le joueur	Chute d'une case par tick
T4	Creuser + garde tombe dans le trou	Garde <code>bloque=true</code> , réinitialisé au rebouchage
T5	Espace + direction sans MUR en bas	Aucun trou créé
T6	Toucher la sortie avec lingots restants	Déplacement refusé
T7	Tous lingots collectés + sortie	Victoire, score +100, passage niveau suivant
T8	Contact avec un garde libre	−1 vie, <code>reset()</code> de toutes les entités
T9	5 contacts avec des gardes	Game Over, boucle arrêtée
T10	Passer sur un garde en Case.TROU	Aucune collision
T11	Creuser + 3 gardes IA à proximité	Les gardes continuent à se déplacer normalement
T12	Finir le niveau 1 (après 2 s)	Niveau 2 chargé automatiquement
T13	Finir le niveau 2 (après 2 s)	Niveau 3 généré et chargé

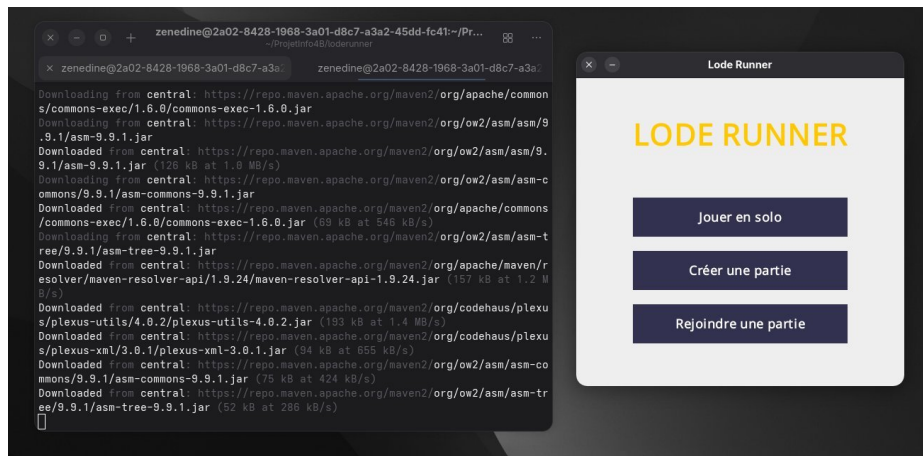


Figure 5: Menu principal : trois modes disponibles (solo, créer une partie, rejoindre). Le terminal à gauche montre la compilation Maven ; la fenêtre Swing à droite affiche le menu.

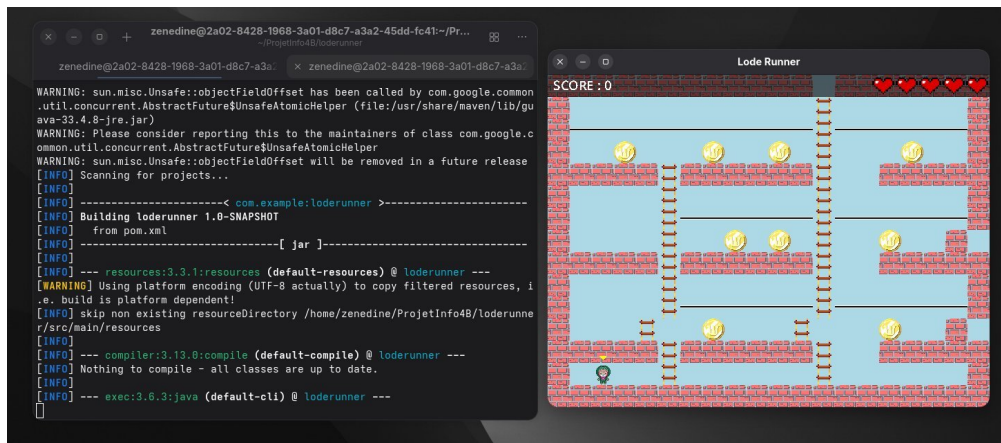


Figure 6: Partie solo en cours (niveau 1) : plateau avec lingots, murs, échelles et passerelles. Le HUD affiche le score (0) et les 5 vies (cœurs). Le joueur est visible en bas à gauche.

6.2 Mode réseau

Test	Action / condition	Résultat attendu
T14	Démarrer le serveur, connecter un client	Rôle et index assignés, plateau affiché
T15	2 clients Joueur connectés	Même état de jeu sur les deux écrans
T16	1 client Joueur + 1 client Garde	Le garde est contrôlable par le joueur distant
T17	Fermer la fenêtre d'un client	Partie continue, pas de crash serveur
T18	Fin de partie (tous morts ou victoire)	Leaderboard affiché + sauvegardé
T19	2 joueurs avec le même nom d'équipe	Score cumulé visible dans classement équipes

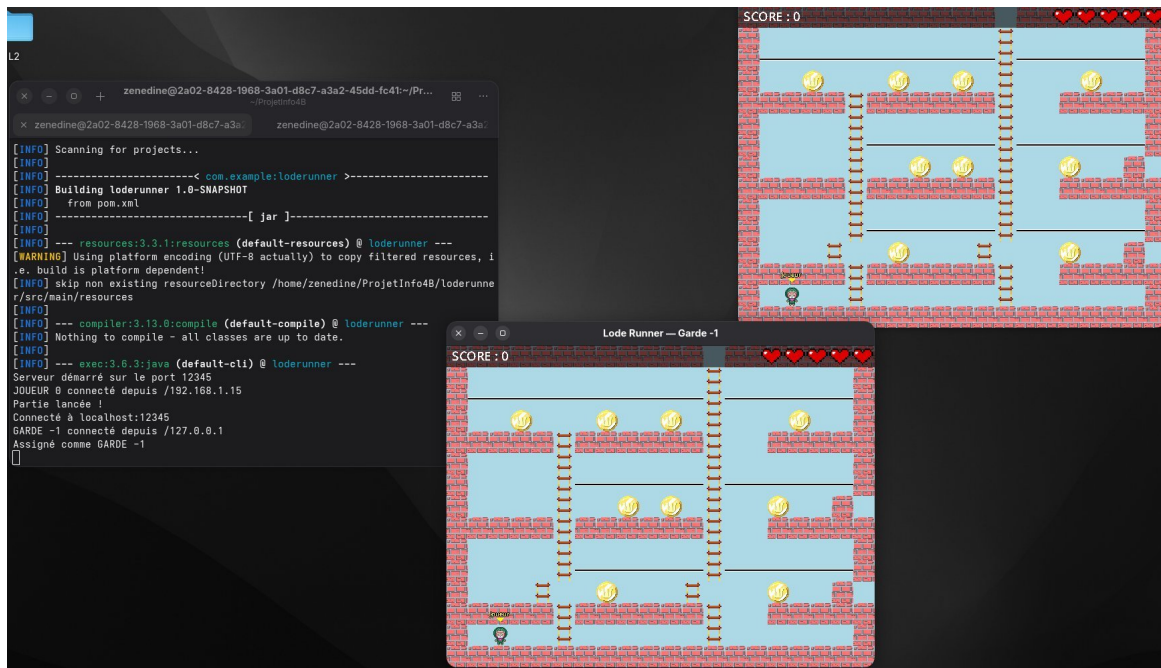


Figure 7: Mode réseau (test 1) : le terminal montre le serveur sur le port 12345, JOUEUR 0 connecté depuis 192.168.1.15, puis GARDE -1 depuis 127.0.0.1 (niveaux 1 et 2 sans garde). Les deux fenêtres Swing affichent le même plateau.

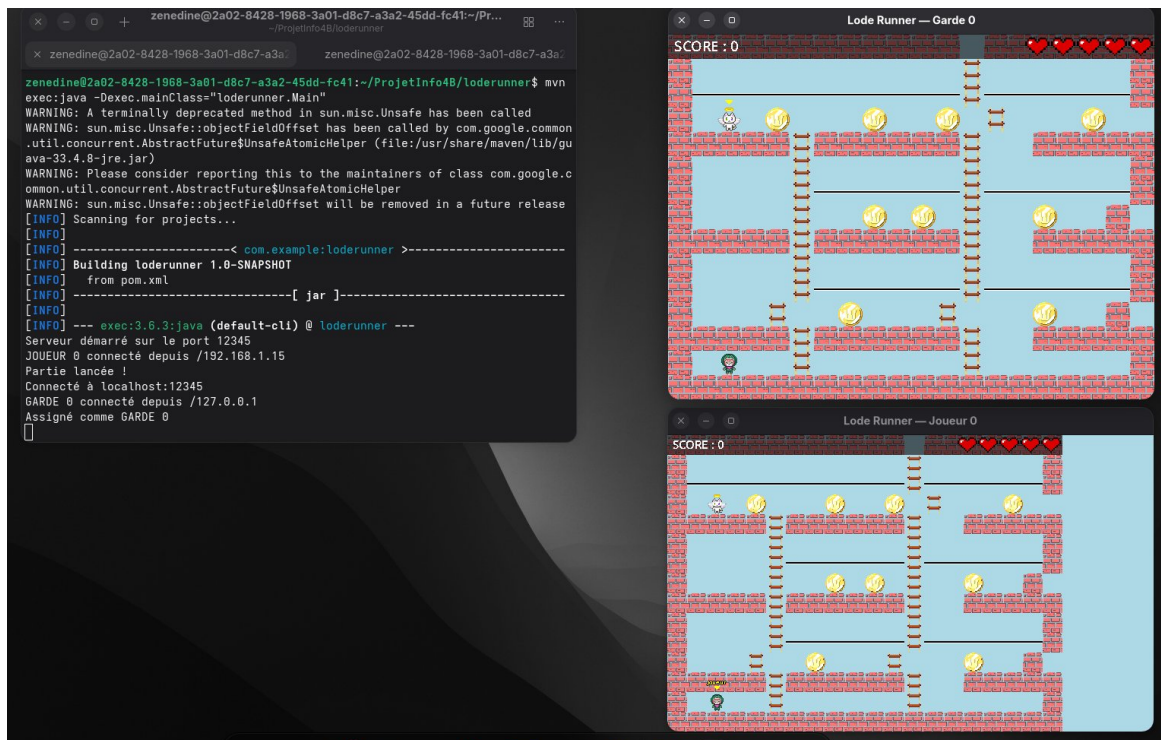


Figure 8: Mode réseau (test 2) : GARDE 0 connecté depuis 127.0.0.1 sur le niveau 3 qui contient un garde. En haut la vue du garde contrôlable, en bas la vue du joueur — les deux clients affichent le même état du jeu en temps réel.

6.3 Génération procédurale

Test	Action / condition	Résultat attendu
T20	Finir le niveau 2	level3.txt créé (20×15)
T21	Rejouer plusieurs fois	Cartes différentes à chaque génération
T22	Rejouer le niveau généré	Lingots accessibles (posés sur sol), sortie atteignable

7 Répartition des tâches

Étudiant	Travaux réalisés
Zenedine EL KARAFI	Modélisation des entités <code>Joueur</code> et <code>Garde</code> ; affichage du classement (<code>LeaderboardWindow</code>) ; définition du protocole de messages (<code>MessageProtocol</code>) ; conception des niveaux de jeu <code>level1.txt</code> et <code>level2.txt</code> ; rédaction du rapport.
Mohamed Kazaliou BAH	Modèle de données (<code>Plateau</code> , <code>Entite</code> , <code>Case</code> , <code>Direction</code>) ; chargement des niveaux (<code>LevelLoader</code>) ; interface graphique (<code>MainWindow</code>) ; saisie clavier (<code>InputHandler</code>) ; ressources graphiques (sprites).
En commun	Mode réseau complet (<code>Serveur</code> , <code>Client</code> , <code>GestionnaireClient</code>) ; moteur de jeu (<code>Moteur</code> , <code>Physique</code> , <code>IA</code> , <code>Regenerateur</code> , <code>GestionnaireNiveaux</code>) ; rendu graphique (<code>GamePanel</code>) ; architecture en couches ; tests et débogage.

8 Conclusion

Ce projet nous a permis d'appliquer concrètement les principes du module Info4B : conception modulaire en couches fonctionnelles, gestion de threads concurrents, synchronisation, communication inter-processus via sockets TCP et persistance des données sur disque.

Les principales difficultés rencontrées ont été l'ordre de création des flux réseau (interblocage de sérialisation), la concurrence dans `Regenerateur` (`Iterator` explicite), la cohérence des chemins relatifs depuis Maven, et le comportement des gardes IA face aux trous (double correction BFS + collision).

Améliorations envisagées : effets sonores (`SoundManager` prévu mais non implémenté), vérification algorithmique de jouabilité pour les niveaux générés, et un vrai éditeur de niveaux.

A Utilisation d'outils LLM

Conformément aux règles du sujet, nous déclarons ci-dessous les lignes de code générées ou corrigées avec l'assistance d'un modèle de langage. Le texte du rapport rédigé avec aide LLM est signalé **en rouge** dans le document.

- **Regenerateur.java, mettreAJour()** – structure `Iterator` :

```
1 Iterator<TrouEnAttente> it = trous.iterator();
2 while (it.hasNext()) { ... it.remove(); }
```

Prompt : « J'ai une ConcurrentModificationException quand je supprime des éléments d'une ArrayList pendant une boucle for-each en Java, comment corriger ? »

- **ImageLoader.java**, `genererImageErreur()` – intégralité de la méthode (carré rose de remplacement).

Prompt : « Comment créer une `BufferedImage` de remplacement en Java quand un fichier image est introuvable ? »

- **GamePanel.java**, `dessinerScoreboard()` – mise en page du tableau des scores multi-joueurs.

- **IA.java**, `peutSeDeplacer()` et `estSupporte()` – correction du bug de figement des gardes :

```
1 // Avant (bug) : caseCible == Case.TROU bloquait le BFS
2 // Après (corrige) : TROU n'est plus filtre dans peutSeDeplacer()
3 // + dans estSupporte() : caseBas == Case.TROU considere comme sol
4 return caseBas == Case.MUR || caseBas == Case.ECHELLE || caseBas == Case
   .TROU;
```

Prompt : « Lorsque je joue si il y a 3 IA et que je creuse un trou, elles se figent toutes jusqu'à ce qu'il se rebouche. »

- **Moteur.java**, `verifierCollisions()` – condition ignorant les gardes en trou :

```
1 if (plateau.getCas(e.g.getX(), g.getY()) == Case.TROU) continue;
```

Prompt : « Quand un garde est figé dans un trou, dès que le joueur se déplace sur lui il perd une vie, tu peux vérifier la condition stp ? »

Le reste du code (architecture en couches, logique de jeu, réseau, génération procédurale, interface graphique, système d'équipes) a été entièrement conçu et rédigé par les membres du groupe.